UNIVERSITATEA TEHNICĂ A MOLDOVEI

FACULTATEA CALCULATOARE, INFORMATICĂ ȘI MICROELECTRONICĂ

DEPARTAMENTUL INGINERIA SOFTWARE ȘI AUTOMATICĂ

Lucrare de laborator nr. 9

Tehnici de programare

Colectii : liste , tupluri , multimi , dictionare

Realizat de : Bulat Madalina

st. gr. SI-262

Verificat de : Emil Cebotaros

asist. univ.

Chisinau 2026

1. Scopul lucrării

Scopul lucrării constă în aplicarea structurilor de date din Python (liste, tupluri, mulțimi și dicționare) pentru colectarea, organizarea și analiza informațiilor despre securitatea unui sistem Linux, identificarea fișierelor cu risc și generarea rapoartelor.

2. Realizarea lucrării

2.1. Partea A – Enumerarea informațiilor despre sistem

A1. Citirea informațiilor din fișiere

Am creat funcția citeste_linii(), care citește informațiile din fișierele suid.txt, sudo.txt și crontab.txt. Funcția elimină liniile goale și comentariile, păstrând doar datele necesare.

A2. Identificarea fișierelor SUID

Am definit mulțimea GTFO, care conține numele programelor ce pot prezenta riscuri de securitate. Am creat funcția suid_constatari(), care identifică fișierele SUID, elimină duplicatele și stabilește nivelul de risc pentru fiecare fișier.

A3. Analiza regulilor SUDO

Am creat funcția sudo_constatari(), care citește regulile SUDO, extrage utilizatorii și drepturile acestora și identifică regulile care conțin NOPASSWD, clasificându-le ca având risc mare.

A4. Analiza sarcinilor CRON

Am implementat funcția cron_constatari(), care citește sarcinile programate, extrage utilizatorii și comenzile executate și organizează informațiile într-o listă de dicționare.

A5. Generarea raportului

Am creat funcția enumereaza(), care grupează rezultatele obținute într-un dicționar. Am utilizat biblioteca json pentru salvarea informațiilor în fișierul enum.json.

La executarea programului au fost identificate 13 fișiere SUID, dintre care 4 cu risc mare, 3 reguli SUDO, dintre care 2 cu risc mare, și 3 sarcini CRON, fără risc ridicat.

2.2. Partea B – Compararea fișierelor SUID

B1. Citirea căilor fișierelor

Am creat funcția cai(), care citește căile fișierelor dintr-un fișier și le transformă într-o mulțime (set). Astfel, sunt eliminate duplicatele și datele pot fi comparate.

B2. Compararea mulțimilor

Am implementat funcția compara(), care utilizează operația de diferență dintre două mulțimi pentru a identifica fișierele SUID noi, care nu se regăsesc în baza de referință.

B3. Identificarea fișierelor periculoase

Am creat funcția analizeaza(), care clasifică fișierele noi în funcție de nivelul de risc, utilizând mulțimea GTFO. Am separat fișierele considerate periculoase de cele cu risc normal.

Au fost identificate următoarele fișiere periculoase noi:

- /bin/cp
- /usr/bin/find
- /usr/bin/nmap
- /usr/bin/vim.basic

B4. Testarea funcționalității

Am creat fișierul test_diferenta.py, în care am implementat testul test_doar_ce_e_nou(). Acesta verifică dacă operația de diferență dintre două mulțimi returnează doar elementele noi.

Am executat testul cu ajutorul comenzii pytest -q test_diferenta.py, iar rezultatul obținut a fost 1 passed, ceea ce confirmă trecerea testului automat.

3. Concluzii

În cadrul acestei lucrări am aplicat structurile de date din Python pentru colectarea, organizarea și analiza informațiilor despre securitatea unui sistem Linux.

În partea A am realizat enumerarea fișierelor SUID, analiza regulilor SUDO și a sarcinilor CRON, clasificarea nivelului de risc și generarea unui raport JSON.

În partea B am utilizat mulțimile pentru compararea fișierelor SUID cu baza de referință, identificarea fișierelor noi și clasificarea celor periculoase. De asemenea, am verificat funcționalitatea prin intermediul testării automate cu Pytest.

Scopul lucrării a fost atins, iar exercițiile realizate au contribuit la consolidarea cunoștințelor despre utilizarea listelor, mulțimilor și dicționarelor în Python.